

Biomedical Data Analytics and AI Report

Sanjana Muppasani (220249243)

1 Research Question

Do protein sequences form distinct communities of homologues?

1.1 Methods

The data is loaded using the *lzma* library. A BLOSUM62 substitution matrix (Henikoff and Henikoff, 1992) was initialized to align amino acid sequences and capture evolutionary relationships, a capability lacking in Hamming distance or simple identity matrix. A **global** alignment algorithm (Needleman and Wunsch, 1970) is used to generate a distance matrix which is then parsed into an undirected graph where edges connect sequences less than a selected distance threshold. Network topology (e.g., density, diameter, transitivity, average path length) was analysed and the *Infomap* algorithm was applied to detect communities.

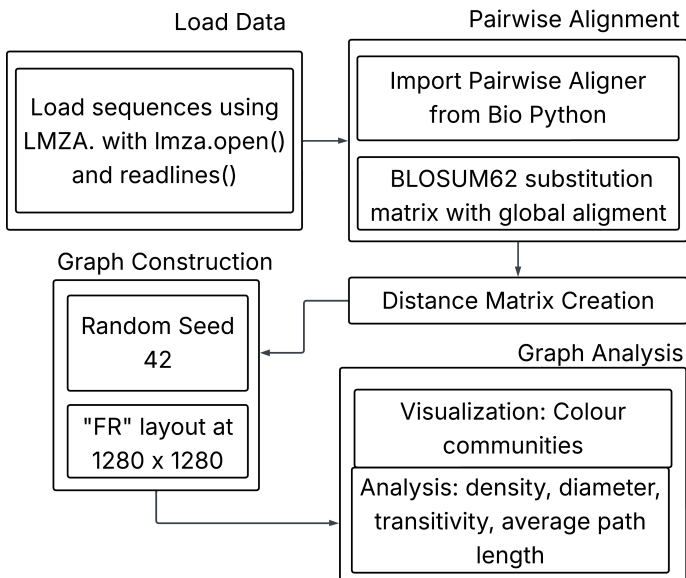


Figure 1: Methodology

1.2 Results

Threshold	Edge Count	Density	Connected Components	Avg. Path Length
0.60	1860	0.014163	229	2.500595
0.70	6388	0.048642	130	1.910969
0.76	10018	0.076282	64	2.038383
0.80	12232	0.093141	29	1.659398
0.90	22049	0.167893	3	2.851199

Table 1: Network Topology Metrics

Table 1 summarizes the network metrics received at various thresholds for a standard node count (513). At very high distances (0.9) the network is extremely dense leading to only 3 connected components. All sequences become part of three interconnected group, offering no meaningful insights. At low distances such as (0.60) the graph has fewer edges and begins to fragment and disintegrate. The edge density decreases and the number of components increases drastically. This

points to a very strict distance threshold.

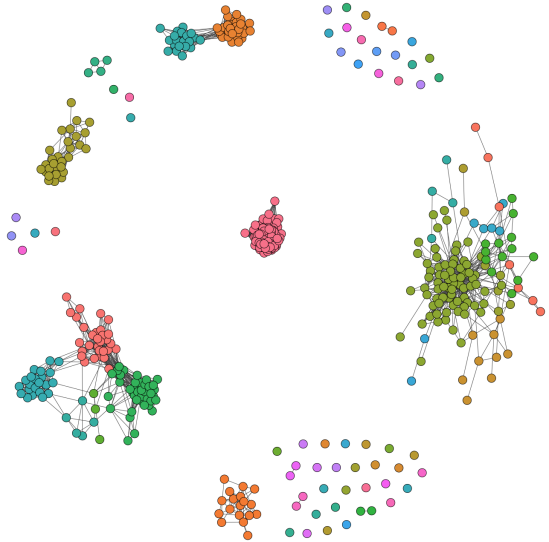


Figure 2: Network at distance threshold 0.76

However, from Figure 2, we observe that at threshold 0.4 the graph is split into 64 distinct components. The number of edges and density indicate good connectivity yet high modularity. the presence of a larger cluster at the far right, clusters with sub-families on bottom left, smaller isolated clusters at top center and 2 diverging families at the top center show a good balance of relationships between classes.

1.3 Interpretation

The analysis of the sequence similarity network confirms that protein sequences organize into distinct communities of homologues rather than a uniform continuum.

Topological metrics reveal a clear structural transition at a distance threshold of 0.76, where the network partitions into 64 discrete components, successfully separating noise from signal. The visual graph supports this, displaying dense, highly interconnected "cliques" that represent conserved families.

The structure is also hierarchical: while many communities are isolated, others form larger super-families linked by "bridging" sequences. This indicates that while distinct homologous groups exist, they also preserve evolutionary relationships, with intermediate sequences connecting divergent sub-families.

1.4 References

- Henikoff, S. and Henikoff, J.G. (1992) 'Amino acid substitution matrices from protein blocks', *Proceedings of the National Academy of Sciences*, 89(22), pp. 10915–10919.
- Needleman, S.B. and Wunsch, C.D. (1970) 'A general method applicable to the search for similarities in the amino acid sequence of two proteins', *Journal of Molecular Biology*, 48(3), pp. 443–453.

2 Research Question

Is it possible to predict the worsening of the dementia rating after the first visit?

2.1 Methods

Data Pre-Processing

All visit information is concatenated into a single dataframe. The data is then cleaned by fixing typos in CDR column, encoding the SEX attribute, and imputing the ASF, SES, MMSE, and eTIV attributes with median and mean values. Data types are assigned to the variables, and ordinal attributes such as CDR and SES are defined.

After pre-processing, the target variable `worsened` is created. A new column `CDR_numeric` is created based on the CDR (Morris, 1993) column assigning numeric values to strings (e.g., 'none' to 0.0). The CDR score from the first visit is the baseline. To accurately capture disease progression, the target variable `worsened` was defined by comparing the baseline CDR (Visit 1) against the CDR score from the patient's final recorded visit. A patient was classified as 'worsened' (Class 1) only if their final CDR strictly exceeded their baseline CDR. Plot the disease progression per visit based on the `CDR_numeric` score calculated.

Machine Learning

Logistic Regression (LR) model is used with random seed 31. All patient visits after 1 are dropped to address the research question. Initialize the columns `sex`, `age`, `YOE`, `SES`, `MMSE`, `eTIV`, `nWBV`, `ASF` and `CDR_numeric` as features and the target as `worsened`. Initialize cross validation (5 splits) with a random seed (31) and Stratified K fold to ensure that the model is tested across all samples of the imbalanced dataset.

For the first part of the experiment, a LR model without penalties is tested using the following hyper parameters: `solver='lbfgs'`, `max_iter=1000`, `C=1.0`, `penalty=None`. The accuracy, 95% confidence interval and f1 scores are measured.

For the next part of this experiment, an LR model with 2 different penalties is used and the performance is compared using the same metrics.

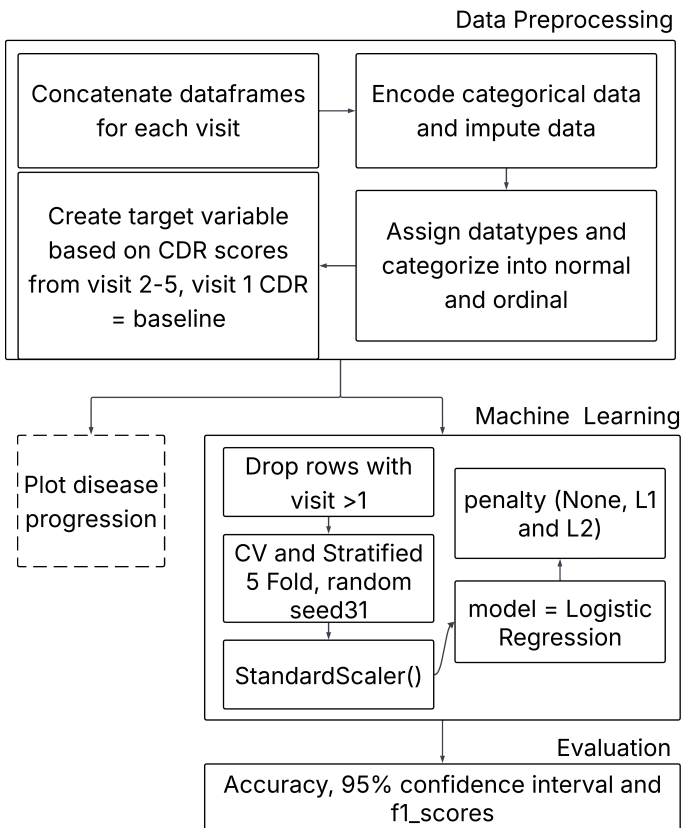


Figure 3: Methodology

2.2 Results

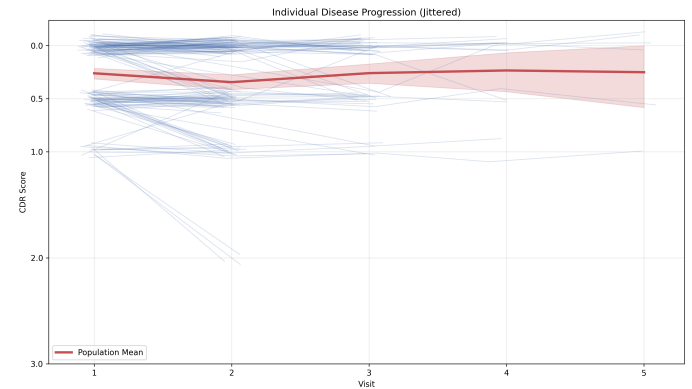


Figure 4: Disease Progression based on CDR_numeric Score

Figure 4 reveals a distinct divide between a stable majority (CDR 0.0) and a rapidly declining sub-population. While the mean CDR suggests slight stabilization between Visits 1–2 followed by a significant worsening through Visit 5, the widening 95% confidence interval indicates decreasing certainty in this average over time

Penalty	Solver	Accuracy	95% CI	F1 Score
None	lbfgs	0.7533	0.6972 - 0.8049	0.0300
L1	liblinear	0.7933	0.7624 - 0.8243	0.0444
L2	lbfgs	0.7733	0.7424 - 0.8043	0.0000

Table 2: Model Performance metrics with different penalties. The Scores given above are the Mean CV scores

Table 2 summarizes the performance of the Logistic Regression models with and without penalties. The model achieved the highest accuracy with L1 regularization at 79.33%. This suggests that the model is good at predicting patients who do not worsen. It also has the narrowest CI ($0.8243 - 0.7624 = 0.062$), suggesting its accuracy estimate is the most precise among the three. However, all models **critically fail** to predict patients who worsen after the first visit. This is reflected in the F1 score which is an important metric to consider for unbalanced classes. The F1 score of 0.04 indicates the model's complete inability to identify a single case of dementia worsening.

2.3 Interpretation

The current architecture fails to predict dementia worsening after the first visit. Despite high accuracy (79%), the F1 score of 0.04 indicates the model failed to learn the minority class, defaulting to a 'majority class' prediction strategy. This confirms that Logistic Regression is a poor model choice for imbalanced data that is not handled with appropriate techniques.

2.4 References

Morris, J.C. (1993) 'The Clinical Dementia Rating (CDR): current version and scoring rules', *Neurology*, 43(11), pp. 2412-2414.

3 Research Question

What combination of algorithms and hyper-parameters can best predict who is likely to miss their appointment?

3.1 Methods

The appointment and visit dataframes are merged and filtered to only include patients who have at least 5 scheduled visits. The health history and appointment patterns are preserved to improve model performance. A data processing pipeline automatically processes categorical data using *OneHotEncoding* and the numeric columns are scaled using the Standard Scaler to ensure mean = 0 and Standard Deviation = 1. However, the *status* is mapped manually to ensure "no-shows" are encoded to 1 instead of 0. Initialize the random state (seed 31) and Stratified K Fold Cross Validation (for the imbalanced dataset) with 10 splits (train size = 90%). Perform a grid search for selected classifiers (KNN, Random Forest and Gradient Boosting (Friedman, 2001)) to identify the best hyper parameters for each of the models. Evaluate the selected models with the best hyper-parameters (Accuracy, F1 Scores, Precision and Recall) to identify the best combination of model and hyper-parameters. Plot the Precision-Recall curves for the models.

Ensemble methods such as Stacking (with Pass-through) and Weighted Soft Voting are also tested using the Gradient Boosting and Random Forest Classifiers. KNN is eliminated as it produces the lowest results. Pass-through is added to ensure that final classifier also makes a prediction based on the dataset. Weighted Soft Voting is used to ensure that the model with higher performance is given more priority and that confidence of the model is considered rather than its guessing capability. Measure the same evaluation metrics.

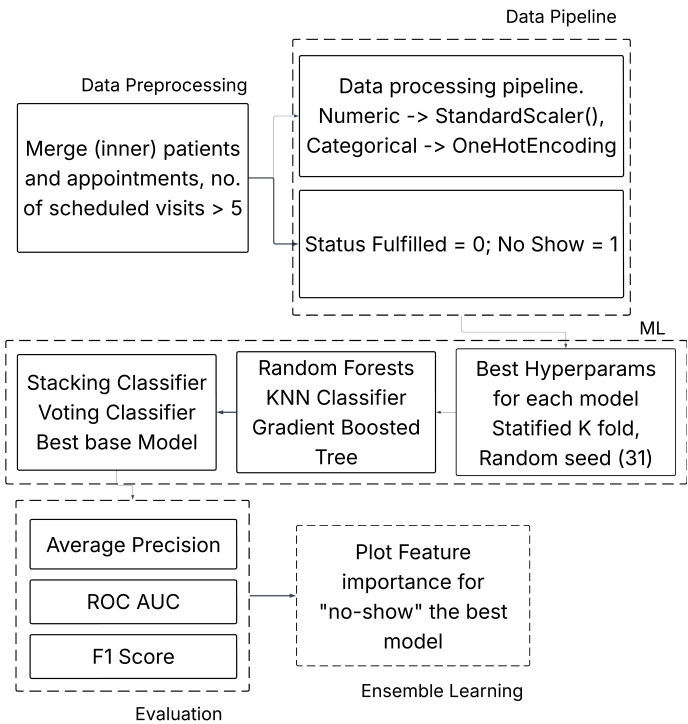


Figure 5: Methodology

3.2 Results

Model	F1 Score	Accuracy	Precision	Recall
KNN	0.28037	0.773809	0.409539	0.213552
Gradient Boosting	0.27368	0.799873	0.547381	0.183026
Random Forests	0.000	0.793184	0.0000	0.0000

Table 3: Average performance metrics for best hyperparameters. The precision and recall are for "No-Shows"

the best performing model. While the F1 score (27.4%) is significantly lower than the accuracy (80%), this discrepancy is expected in highly imbalanced datasets.

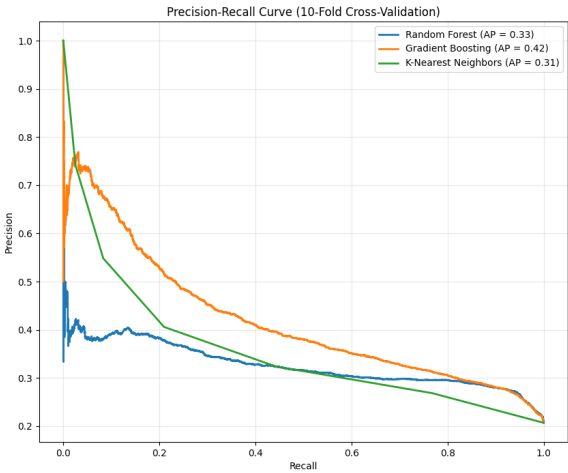


Figure 6: PR Curves for Different Models

The Precision-Recall Curves (Figure 6) show that while all models outperform the baseline (0.20), the GB Classifier exhibits superior robustness (AP = 0.42), maintaining higher precision across recall thresholds. KNN initially exhibits high precision however the scores degrade for increasing recall and the Random Forest is unstable for low recall levels. The Gradient Boosted Classifier offers the most robust performance out of all models tested, accounting for some class imbalance.

Model	Average Precision (AP)	ROC AUC	F1 Score
Weighted Soft Voting	0.4273	0.742047	0.184484
Standalone GB (Baseline)	0.416014	0.734412	0.260285
Stacking (Passthrough)	0.393603	0.727745	0.0716533

Table 4: Ensemble Methods and Standalone Model Comparison

Gradient Boosting maintained the best balance of metrics compared to other ensemble models, securing the highest F1 score and the highest Precision overall (0.55), proving it to be more robust than Stacking or Voting.

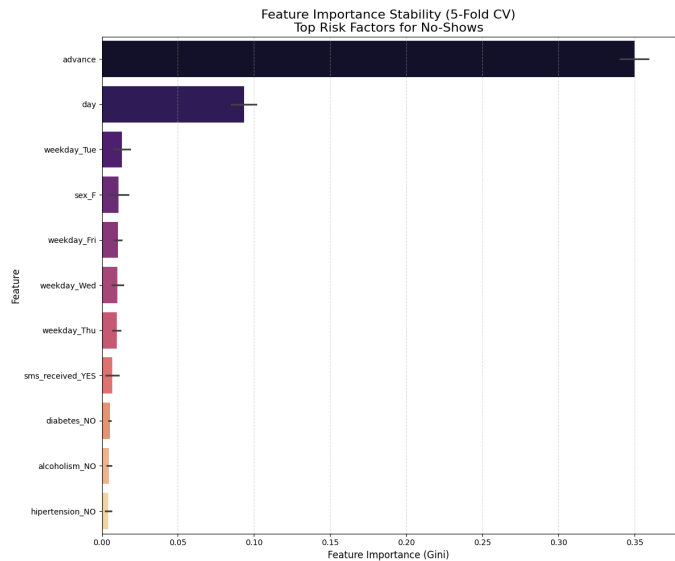


Figure 7: Feature Importance for GB

Table 3 identifies the Gradient Boosted Classifier (GB) as one of

Figure 7 displays the Gini importance, identifying the advance attribute as the dominant feature in determining if a patient will miss their appointment. All the features have a short errorbar indicating they are of equal importance across folds.

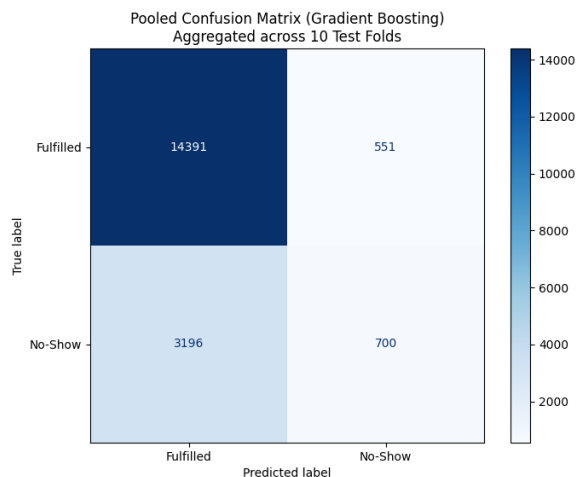


Figure 8: Pooled Confusion Matrix

Figure 8 shows that Gradient Boosted Classifier correctly identified the "fulfilled" cases, however critically failed in identifying "no shows". It misclassified 3165 out of 3896 values.

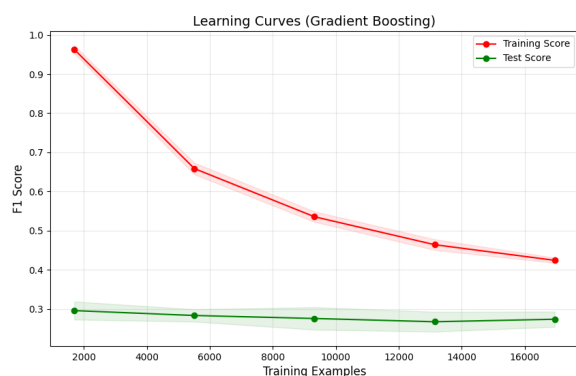


Figure 9: Learning Curves

Figure 9 shows the dropping of the Training F1 scores as the training set increases (0.96 to 0.42). This is indicative of high bias. The model also shows an inability to generalize as it hits a performance "ceiling" due to the lack of predictive power of the features. The convergence of the Training and Validation scores shows that the model is not complex enough to capture the underlying patterns.

3.3 Interpretation

Gradient Boosting is a good model identified to predict the "no-shows" for appointments. To further improve the performance of the model, class imbalances should be addressed. To ensure less number of patients miss the appointment, reminders should be sent to people who have a long wait time before their appointment.

3.4 References

Friedman, J.H. (2001) 'Greedy function approximation: a gradient boosting machine', *Annals of Statistics*, pp. 1189-1232.

4 Research Question

How well can we predict the protein function from its sequence?

4.1 Methods

The domain classification, S60 sequences and super family names are loaded and the information from the domain_classification file for the categories C A T H is parsed into a sequence ID in the format C.A.T.H. The sequences and domains are merges on domain_id and top 5 superfamilies with less than 1000 sequences are filtered.

The sequences are tokenized on a character lever and padded with zeroes to the maximum length of sequences and the superfamilies selected are encoded.

Keras Tuner is to find the best architectures (units, learning rates and dropouts) for 1D CNN, LSTM and GRU. Hyper-parameters were optimized using the Hyperband algorithm (Li et al., 2017). The best performing model is then evaluated with a stratified and nested CV loop to eliminate any selection bias by ensuring the final performance is calculated on data that was not used for hyperparameter tuning.

Models were configured for 50 epochs using the "Adam" optimizer. The sparse categorical cross-entropy loss function was selected to process the integer-encoded superfamily labels. To prevent overfitting, an Early Stopping protocol halted training if validation loss did not improve within 10 epochs. Model performance (Accuracy and Loss) was plotted over increasing epochs. The experiment was then replicated for the S40 dataset.

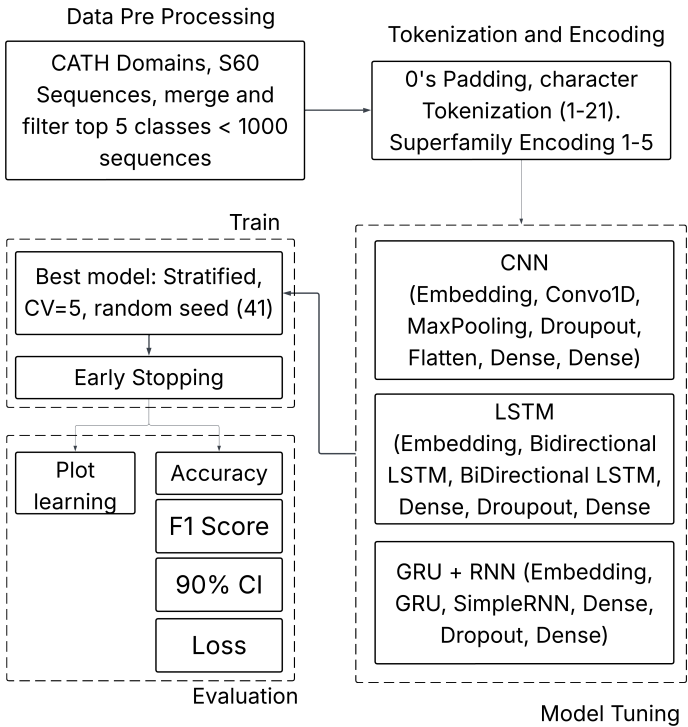


Figure 10: Methodology

4.2 Results

Model	Units	Learning Rate	Dropout	Accuracy
CNN	128	0.001	0.5	83%
LSTM	64	0.01	0.5	72%
GRU + RNN	32	0.01	0.2	79%

Table 5: Best models with different layer types using Keras Tuner

Based on Table 5, there is a clear hierarchy of the models based on their performance, the CNN shows the highest accuracy of 81% followed by GRU+RNN (79%) and LSTM(72%) respectively.

Accuracy	Macro F1 Score	95% CI	Loss
83.22%	81.94%	3.90%	1.15

Table 6: Results of the best performing model: 1D CNN (without early stopping)

Table 6 reports an accuracy of 83.22%, confirming the model's effectiveness on unseen data. While the Macro F1 score ensures balanced evaluation across classes, fluctuations in cross-entropy loss indicate that prediction confidence varies significantly between folds despite stable classification accuracy.

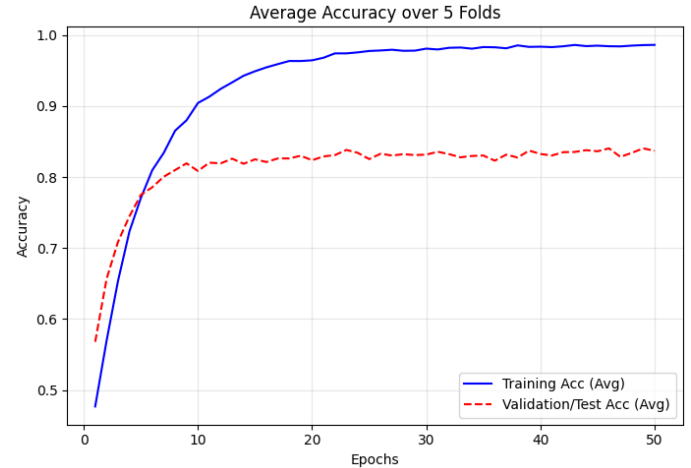


Figure 11: Accuracy of CNN (50 epochs, no early stopping)

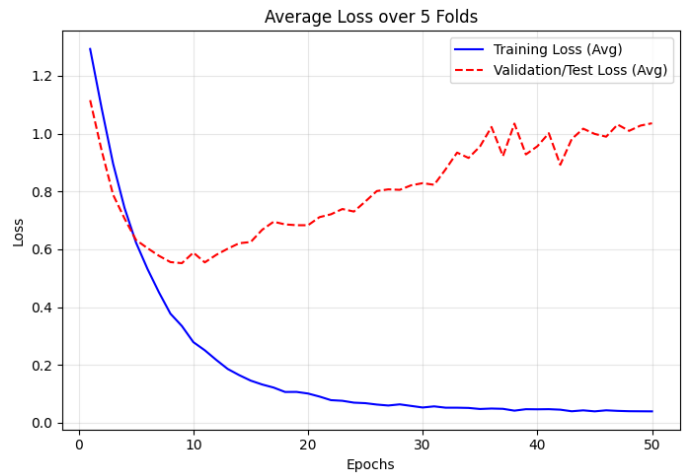


Figure 12: Loss of CNN (50 epochs, no early stopping)

Based on Figure 11, you can see that the model runs rapidly over the first 10 epochs and hits a ceiling after 10 epochs. It can be observed from Figure 12 that the loss graph reaches an "elbow point" around 10 epochs and then begins to rise indicating overfitting.

Accuracy	Macro F1 Score	95% CI	Loss
81.05%	79.43%	2.19%	0.55

Table 7: Results of 1D CNN with Early Stopping

Adding an early stopping mechanism addresses overfitting based on the information from Table 7. The loss value decreases significantly. While this also reduces the Accuracy and the Macro F1 score, this is a tradeoff that needs to be made to mitigate overfitting.

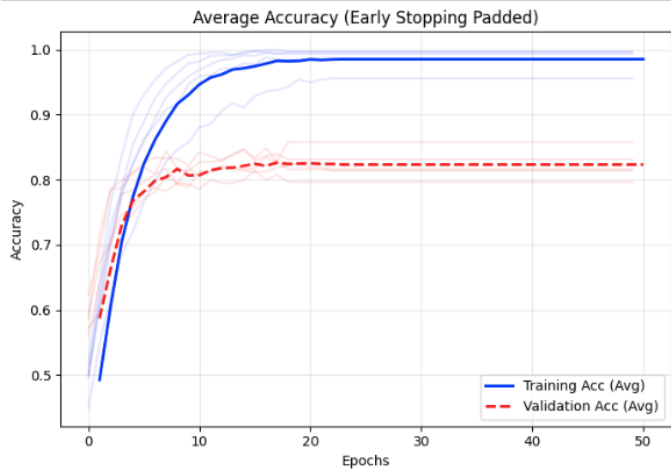


Figure 13: Accuracy of CNN (with early stopping)

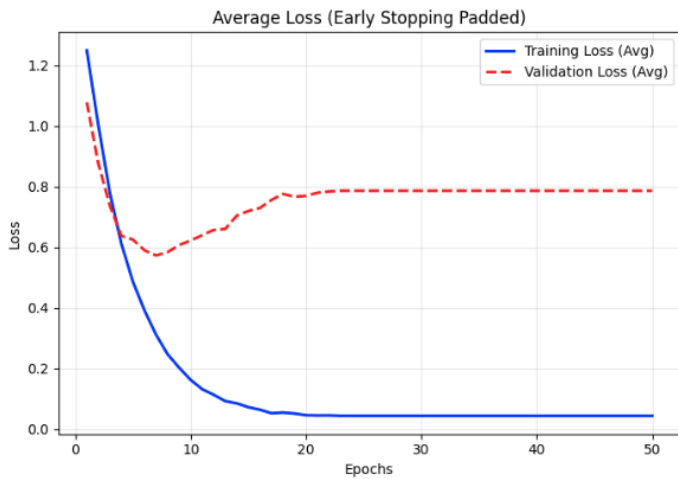


Figure 14: Loss of CNN (with early stopping)

Figure 13 and 14 show individual faint lines in the above graph indicating that different folds stopped at different times, however the bold blue and red lines show the stabilization effect of early stopping. While the loss increased slightly from its minimum at 7 epochs, this allowed prevented overfitting while maintaining a good accuracy score.

The same experiment run with S40 dataset produces the following results:

Accuracy	Macro F1 Score	95% CI	Loss
81.37%	76.67%	3.24%	0.51

Table 8: Results of 1D CNN for S40 dataset (with Early Stopping)

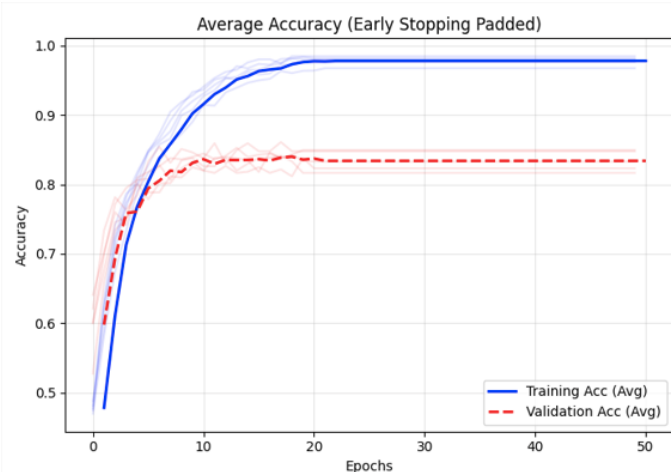


Figure 15: Accuracy of CNN for S40 dataset (with early stopping)

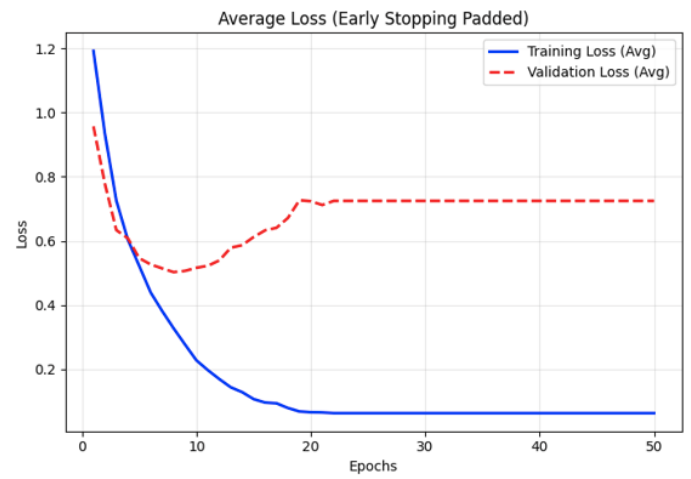


Figure 16: Loss of CNN S40 dataset (without early stopping)

Contary to typical homology benchmarks, the above model (as shown by Table 8, Figure 15 and Figure 16) exhibits negligible degradation in performance from the S60 dataset.

4.3 Interpretation

We can predict protein function from sequence with a high degree of reliability. The best-performing model (1D-CNN) achieved a generalization accuracy of 83.22% ($\pm 3.52\%$). This indicates that the raw amino acid sequence contains sufficient signal to correctly categorize the protein's biological function (majority of cases) and the protein function prediction task benefits more from the spatial pattern detection capabilities of CNNs than from the temporal memory mechanisms of RNNs.

4.4 References

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A. and Talwalkar, A. (2017) 'Hyperband: A novel bandit-based approach to hyperparameter optimization', *The Journal of Machine Learning Research*, 18(1), pp. 6765-6816.

5 Research Question

How well can a deep neural network classify retinal images to identify eye disorders?

5.1 Methods

The data is loaded into a train, validation and test set using `image_dataset_from_directory`. The training dataset is shuffled to prevent potential overfitting. Images are resized to (150 x 150 x 3) to reduce computational costs (due to hardware limitation) and pixel values are rescaled to [0,1] to ensure numerical stability during gradient descent.

The custom CNN layer comprised of stacked blocks of `Conv2D` (using `ReLU` activation) and `MaxPooling2D` layers, followed by `Flatten` and `Dense` layers. To optimize the architecture, the `keras_tuner` package with the `Hyperband` algorithm was employed. The search space included **filters (38-128)** dropout rates **(0.2-0.5)**, and learning rates. The model is compiled with the `Adam` optimizer and Categorical Cross Entropy.

The trained model is then used to make predictions on the hold out test set. Beyond the Loss and Accuracy, we also calculate the Macro F1 score to account for any potential imbalance in dataset for the multi-class classification problem. The confusion matrix and learning curves (Accuracy and Loss) are plotted.

The custom CNN was benchmarked using Transfer Learning via MobileNetV2 (Sandler et al., 2018) initialized with ImageNet weights to mitigate potential data scarcity. Inputs were normalized to [-1,1]. The classification layers are replaced with a `GlobalPooling` layer to collapse dimensions, followed by a `Dense` and `Dropout` layer to enforce regularization before the final `Softmax` function is applied. Training occurred in 2 stages: freezing base weights for 20 epochs, followed by fine-tuning top layers.

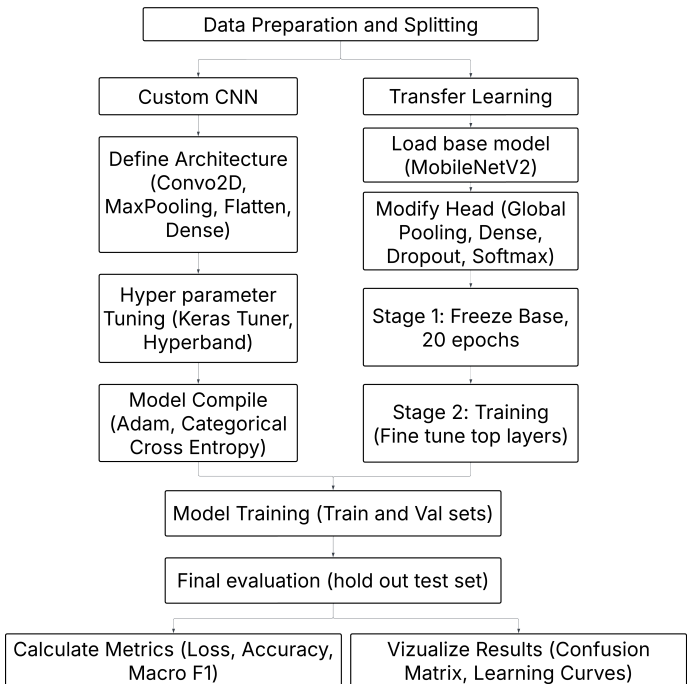


Figure 17: Methodology

5.2 Results

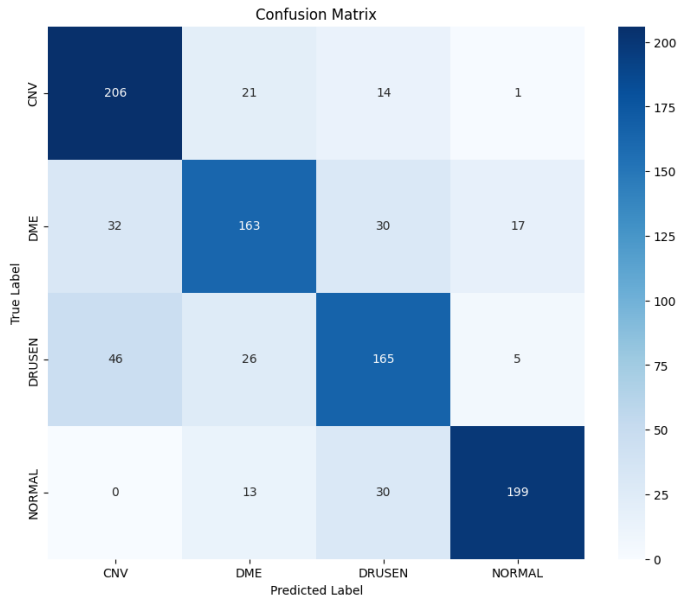


Figure 18: Confusion Matrix for Custom CNN

Based on the confusion matrix from Figure 18, it can be observed that the custom CNN is particularly good at identifying healthy eyes, however the model struggles learning patterns for diseased.

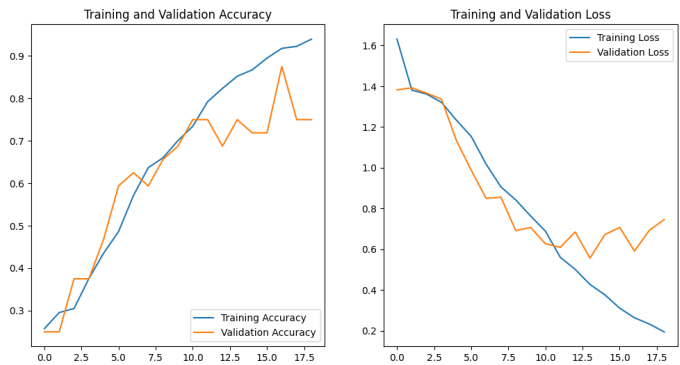


Figure 19: Learning Curves for Custom CNN

Class	Precision	Recall	F1 Score
CNV	73%	85%	78%
DME	73%	67%	70%
DRUSEN	69%	68%	69%
Normal	90%	82%	86%
Weighted Avg	76%	76%	76%

Table 9: Results for Custom CNN

Based on Table 9, the CNN can be used as a good baseline with a high overall performance. (Average F1 score 76%). However, to further support the results from the confusion matrix, the model is great with identifying Normal eyes (F1 86%) however struggles with DME (Recall 61%) and DRUSEN (Precision 69%).

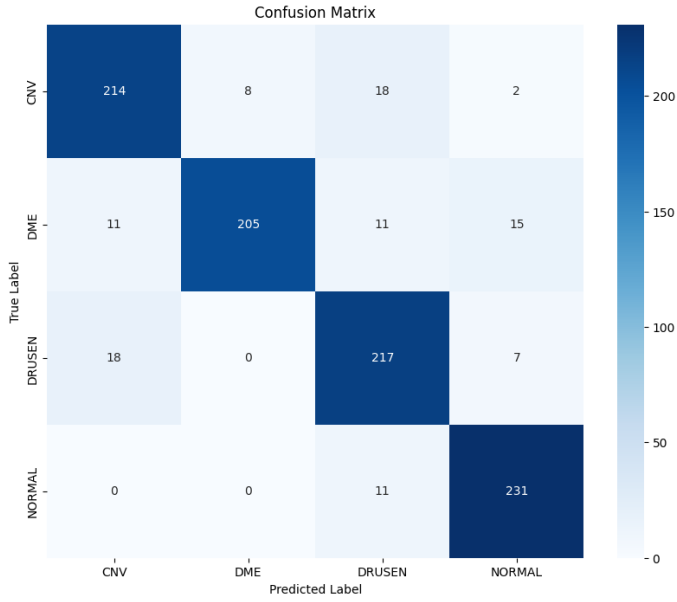


Figure 20: Confusion Matrix for MobileNetV2 (+ custom layers)

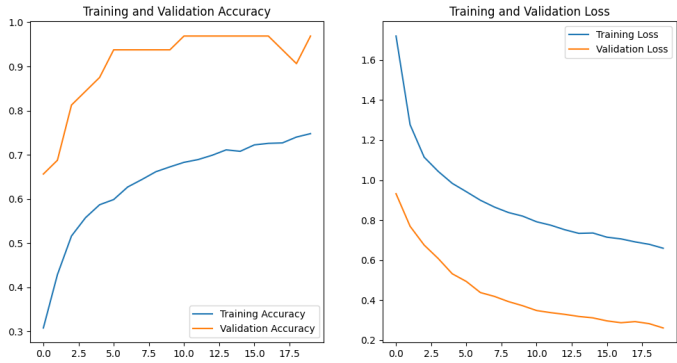


Figure 21: Learning Curves for MobileNetV2 (+ custom layers)

Class	Precision	Recall	F1 Score
CNV	88%	88%	88%
DME	96%	85%	90%
DRUSEN	84%	90%	87%
Normal	91%	95%	93%
Weighted Avg	90%	90%	90%

Table 10: Results for MobileNetV2 (+ custom layers)

Based on the results from Table 10 and Figure 20, the MobileNetV2 Transfer Learning model shows substantial improvement in performance. There is a significant increase in the Weighted Average Score (90%). The model unlike the custom CNN is very good in not just identifying normal eyes from diseased ones, but also specifying the type of disease. DME achieved 96% Precision - which was a problem for CNN. Based on Figure 21, the model also shows a strong higher accuracy at validation than at training due to the effect of dropout layers which are active for testing allowing the model to utilize full predictive capability.

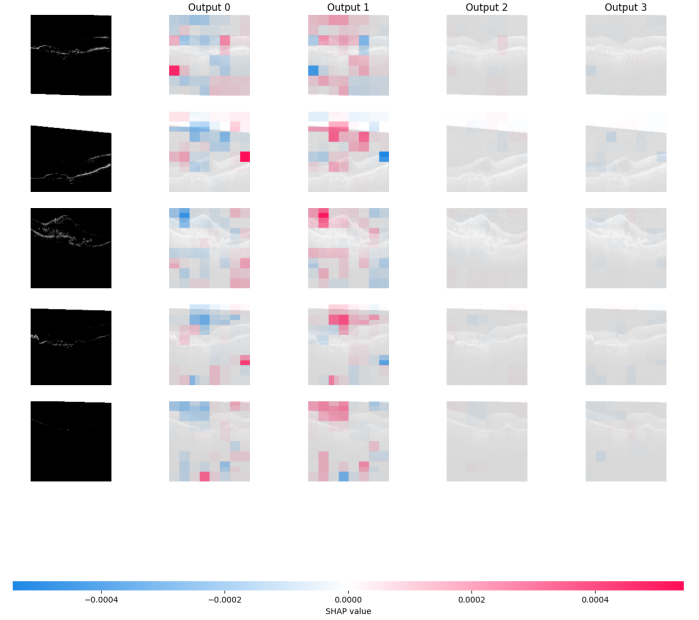


Figure 22: Learning Curves for MobileNetV2 (+ custom layers)

In the OCT scans (Figure 22), columns Output 0–3 represent disease cases. Red blocks indicate features increasing class probability, while blue blocks reduce it. The visualization shows correct classification into Class 1, with activation concentrated on tissue matter, confirming the model relies on relevant biological features rather than artifacts.

5.3 Interpretation

A custom CNN can be used to rule out eye conditions however it is not effective for differentiating between different diseases. Transfer learning models can be used to classify retinal images **extremely well**, serving as a highly reliable method for automated screening.

5.4 References

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.C. (2018) 'MobileNetV2: Inverted residuals and linear bottlenecks', in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510-4520